

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
Facultad de Ciencias de Computación • Carrera de Ingeniería en Software

Arquitectura de Microservicios con API Gateway: Análisis del Caso Netflix y Aplicación a un Sistema de Préstamos

Asignatura: Administración de Plataformas
Docente: Ing. Guerrero Ulloa Gleiston Ciceróne
Estudiante: Luna Mora Ernesto Gregory
Curso: Sexto Semestre – Ingeniería en Software

Resumen—Este trabajo analiza la migración de Netflix hacia una arquitectura de microservicios con API Gateway como caso de estudio paradigmático de la industria. Se examinan las motivaciones técnicas y de negocio, los beneficios obtenidos (escalabilidad horizontal, despliegue continuo, resiliencia) y los desafíos reales afrontados (consistencia eventual, latencia inter-servicio, observabilidad distribuida). Finalmente, se proyectan los aprendizajes al diseño de un sistema propio de préstamos y gestión de pagos, argumentando decisiones arquitectónicas concretas.

I. INTRODUCCIÓN Y CASO REAL

En 2008, Netflix sufrió una corrupción crítica de su base de datos relacional monolítica que interrumpió el envío de DVDs durante tres días consecutivos. Ese evento catalizó una de las mayores transformaciones arquitectónicas de la industria: la migración completa hacia microservicios desplegados en Amazon Web Services (AWS), proceso que tomó aproximadamente siete años [2]. Para el año 2015, Netflix operaba más de 500 servicios independientes; actualmente supera los 1 000, atendiendo más de 200 millones de suscriptores en 190 países con picos de más de 2 000 millones de solicitudes diarias.

Como componente central, Netflix desarrolló **Zuul**, su propio *API Gateway* de código abierto, que actúa como único punto de entrada para el tráfico externo. Zuul implementa enrutamiento dinámico, balanceo de carga, autenticación, terminación TLS y limitación de velocidad (*rate limiting*), abstrayendo la topología interna de los servicios frente a los clientes [3]. Complementariamente, la biblioteca **Hystrix** implementó el patrón *Circuit Breaker* para aislar fallos, mientras que **Eureka** proporcionó descubrimiento de servicios.

II. JUSTIFICACIÓN TÉCNICA Y DE NEGOCIO

Las razones que motivaron la decisión de Netflix responden a dos dimensiones:

Razones de Negocio

- **Velocidad de innovación:** el monolito impedía lanzar nuevas funcionalidades sin congelar el sistema completo. El modelo de microservicios permite que equipos pequeños (*two-pizza teams*) iteren de forma autónoma.
- **Disponibilidad global:** el SLA de Netflix exige 99,99 % de disponibilidad; la arquitectura distribuida permite recuperación ante fallos sin paradas globales.
- **Elasticidad económica:** escalar sólo los servicios bajo demanda (v.g. codificación de vídeo en horas pico) reduce costes operativos en AWS respecto a escalar el monolito completo.

Razones Técnicas

- **Acoplamiento rígido:** cualquier cambio en el monolito requería compilación, prueba y despliegue del sistema entero, lo que elevaba el riesgo de regresiones [1].
- **Escalabilidad selectiva:** los microservicios permiten asignar recursos de cómputo según la carga real de cada función (streaming, recomendaciones, autenticación), aplicando el principio

de *Single Responsibility* a nivel arquitectónico [2].

- **Heterogeneidad tecnológica:** cada servicio puede usar el lenguaje, base de datos o framework óptimo para su dominio, sin imponer restricciones al resto del sistema.

III. BENEFICIOS Y DESAFÍOS

Beneficios Obtenidos

1. **Escalabilidad horizontal independiente:** Netflix escala de forma dinámica sólo los servicios que lo requieren, logrando reducir el tiempo de respuesta promedio por debajo de 50 ms para la API de catálogo incluso en picos de tráfico [4].
2. **Despliegue continuo (CI/CD):** con microservicios, Netflix realiza miles de despliegues diarios sin interrupciones de servicio, habilitando prácticas de *canary releases* y *blue-green deployments* para validar cambios en producción de forma gradual [1].
3. **Resiliencia y aislamiento de fallos:** el patrón *Circuit Breaker* (Hystrix) detecta servicios degradados y activa respuestas de contingencia (*fallbacks*), evitando la propagación de fallos en cascada que afectaría a millones de usuarios simultáneos.
4. **Centralización de funciones transversales mediante API Gateway:** Zuul concentra autenticación, registro de auditoría (*logging*), trazado y transformación de protocolos, eliminando la duplicación de esta lógica en cada microservicio [3].
5. **Políglotas tecnológicos:** equipos especializados en recomendaciones usan Python/TensorFlow, mientras los servicios de streaming usan Java/Scala, todos tras la misma interfaz de API Gateway.

Desafíos Reales Afrontados

1. **Consistencia de datos distribuida:** al eliminar transacciones ACID entre servicios, Netflix adoptó *consistencia eventual* y el patrón SAGA con orquestación para mantener la coherencia entre el historial de visualización, la facturación y el perfil del usuario [2].
2. **Latencia inter-servicio:** la descomposición amplificó los saltos de red (*network hops*); Netflix respondió con gRPC para comunicaciones internas, caché distribuida EVCache (basada en Memcached) y el patrón *Backend for Frontend* (BFF) en Zuul.
3. **Observabilidad y trazado distribuido:** rastrear una solicitud de usuario a través de 30+ microservicios requirió inversión en Zipkin (trazado), un stack ELK centralizado (Elasticsearch-Logstash-Kibana) y el sistema propietario de métricas Atlas [4].

4. **Complejidad operacional:** gestionar cientos de servicios elevó la deuda de infraestructura; Netflix respondió con Spinnaker (orquestación de despliegues) y Chaos Monkey (*chaos engineering*) para verificar la resiliencia en producción.
5. **Sobrecarga del API Gateway:** Zuul como único punto de entrada puede convertirse en cuello de botella; Netflix lo mitigó con instancias replicadas, autoescalado y la migración a Zuul 2 (asíncronico, basado en Netty), mejorando la capacidad de manejo de conexiones concurrentes.

IV. APLICACIÓN AL PROYECTO PROPIO

Los aprendizajes del caso Netflix se proyectan directamente al diseño de un **Sistema de Préstamos y Gestión de Pagos** con la siguiente arquitectura de microservicios:

Servicio	Responsabilidad
Usuarios	Registro, autenticación y perfiles de clientes
Solicitudes	Recepción y ciclo de vida del préstamo
Riesgo Crediticio	Evaluación del score y aprobación
Pagos	Procesamiento de cuotas y reconciliación
Notificaciones	Alertas por correo/SMS/push
Reportes	Dashboards y estados de cuenta

API Gateway centralizado: se implementará un gateway (Kong o AWS API Gateway) como único punto de entrada que gestione autenticación JWT, control de acceso por roles (RBAC), limitación de velocidad y registro de auditoría, replicando la arquitectura de Zuul en Netflix [3].

Consistencia en pagos: dado que la generación de un préstamo actualiza simultáneamente saldo, tabla de amortización y estado del cliente, se aplicará el patrón **SAGA con orquestación**, garantizando reversión controlada (*compensating transactions*) si algún servicio falla, como solución a los desafíos de consistencia eventual documentados por Netflix [1].

Observabilidad: se adoptará **OpenTelemetry** para trazado distribuido y **Prometheus + Grafana** para métricas operacionales, inspirado en el stack de observabilidad de Netflix, permitiendo identificar cuellos de botella en el flujo de aprobación de préstamos.

Escalabilidad selectiva: el Servicio de Pagos se escalará horizontalmente mediante Kubernetes durante los picos de vencimiento de cuotas (fin/inicio de mes), sin afectar a los demás servicios, replicando el modelo de escalado selectivo que Netflix aplica en picos de tráfico [4].

V. CONCLUSIÓN

El caso Netflix demuestra que la arquitectura de microservicios con API Gateway no es una solución universal, sino un estilo arquitectónico que entrega sus máximos beneficios cuando la escala operacional, la velocidad de innovación y la resiliencia son requisitos fundamentales del negocio [2]. Los desafíos de consistencia, observabilidad y complejidad operacional son reales y exigen madurez de ingeniería. Para el sistema de préstamos propuesto, la adopción de estos patrones —Gateway, SAGA, Circuit Breaker y trazado distribuido— permite proyectar un sistema escalable, seguro y mantenible desde etapas tempranas del diseño.

REFERENCIAS

- [1] A. Balalaie, A. Heydarnoori y P. Jamshidi, “Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architec-

ture,” *IEEE Software*, vol. 33, no. 3, pp. 42–52, may./jun. 2016. doi:10.1109/MS.2016.64 [IEEE Xplore: 7436659]

- [2] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin y L. Safina, “Microservices: Yesterday, Today, and Tomorrow,” en *Present and Ulterior Software Engineering*, M. Mazzara y B. Meyer, Eds. Springer, 2017, pp. 195–216. doi:10.1007/978-3-319-67425-4_12 [Scopus: Indexado; 662+ citas]
- [3] J. T. Zhao, S. Y. Jing y L. Z. Jiang, “Management of API Gateway Based on Micro-service Architecture,” *J. Phys.: Conf. Ser.*, vol. 1087, no. 3, p. 032032, sep. 2018. doi:10.1088/1742-6596/1087/3/032032 [IOP Publishing – CC BY 3.0]
- [4] G. Blinowski, A. Ojdowska y A. Przybyłek, “Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,” *IEEE Access*, vol. 10, pp. 20357–20374, 2022. doi:10.1109/ACCESS.2022.3152803 [IEEE Xplore: 9713226]